Ferramenta Baseada em Linguagem de Domínio Específico para o Apoio ao Ensino de Autômatos Finitos com Simulação Visual Interativa

João Thomaz Vieira Klaus Siegfried Beckmann

Bacharelado em Ciência da Computação
Universidade Tuiuti do Paraná
Orientador: Prof. Diógenes Cogo Furlan

Curitiba, 2026

Roteiro

1. Introdução
2. Fundamentação Teórica
3. Trabalhos Relacionados
4. Metodologia de Construção da Solução
5. Ferramentas para Implementação
6. Projeto da DSL
7. Implementação
8. Resultados e Discussões
9. Conclusão

Roteiro

1. Introdução
2. Fundamentação Teórica
3. Trabalhos Relacionados
4. Metodologia de Construção da Solução
5. Ferramentas para Implementação
6. Projeto da DSL
7. Implementação
8. Resultados e Discussões
9. Conclusão

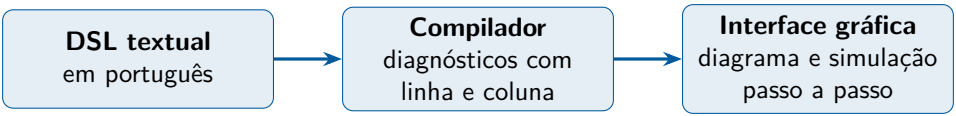
- Autômatos finitos são um pilar da Ciência da Computação, mas a disciplina é **reconhecidamente difícil**.
- Morazán e Antunez (2014): o estudante projeta autômatos **sem poder experimentar** nem receber **retroalimentação imediata**, ao contrário da programação.
- As ferramentas mais difundidas (**JFLAP**) são **só visuais**: sem descrição textual, sem versionamento, **sem diagnóstico** de erros.
- A alternativa textual (**FSM**, em Racket) descreve autômatos como código, mas **não os desenha nem anima**.

Lacuna

Unir a **descrição textual**, o **diagnóstico de um compilador** e a **simulação visual** em uma única ferramenta.

Pergunta de pesquisa

Como uma linguagem textual de domínio específico, processada por um compilador com diagnósticos localizados e acoplada a uma simulação visual, pode oferecer ao estudante **retroalimentação imediata** na construção e na execução de autômatos finitos?



Autosim: o estudante escreve o texto; o diagrama é gerado e animado automaticamente

Objetivos

Objetivo geral: desenvolver uma ferramenta educacional que integra a descrição textual de autômatos, sua compilação pedagógica e sua simulação visual em um único ambiente.

Objetivos específicos

1. Projetar uma **DSL** textual para AFDs e AFNs, com gramática em EBNF;
2. Implementar um **compilador pedagógico** com diagnósticos localizados;
3. Desenvolver um **simulador** de cadeias sobre os autômatos descritos;
4. Construir uma **interface gráfica** que desenhe e anime o autômato;
5. **Validar** a ferramenta com testes, autômatos clássicos e cenários de erro.

Validação **funcional**; a eficácia pedagógica com estudantes é trabalho futuro.

Roteiro

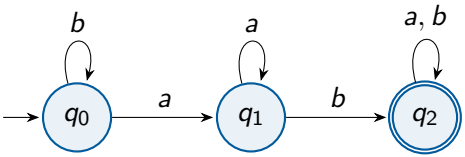
1. Introdução
2. Fundamentação Teórica
3. Trabalhos Relacionados
4. Metodologia de Construção da Solução
5. Ferramentas para Implementação
6. Projeto da DSL
7. Implementação
8. Resultados e Discussões
9. Conclusão

Um **AFD** é uma quíntupla $M = (Q, \Sigma, \delta, q_0, F)$:

- Q: estados Σ: alfabeto
- $\delta : Q \times \Sigma \to Q$: transições
- q_0: inicial $F \subseteq Q$: finais

No **AFN**, $\delta : Q \times (\Sigma \cup \{\varepsilon\}) \to \mathcal{P}(Q)$: vários destinos e transições vazias ε.

AFD e AFN são **equivalentes** (construção de subconjuntos) e reconhecem as **linguagens regulares**.

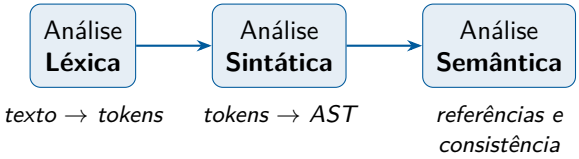


Cadeias que **contêm "*ab*"**

aceita: ab, aab, bab

rejeita: ε, ba, bbb

Reaparece na DSL e na interface.



DSL (Fowler, 2010)

Linguagem de **expressividade limitada**, voltada a um **domínio específico**.

- **Interna:** embutida em uma linguagem hospedeira (ex.: FSM, em Racket).
- **Externa:** sintaxe própria e **compilador dedicado** — é o caso da **Autosim**.

Roteiro

1. Introdução
2. Fundamentação Teórica
3. Trabalhos Relacionados
4. Metodologia de Construção da Solução
5. Ferramentas para Implementação
6. Projeto da DSL
7. Implementação
8. Resultados e Discussões
9. Conclusão

Panorama: quatro ferramentas gráficas e uma textual

- **JFLAP** — referência mundial; cobre toda a hierarquia de Chomsky; construção **só gráfica**, sem descrição textual nem diagnóstico.
- **GAM** (UFLA) — *desktop*; limitado a 10 estados e 3 símbolos.
- **Simulador UFSC** — *web*, sem instalação; construção gráfica.
- **AutomatoGraph** — só AFDs; gera a gramática regular; simulação parcialmente visual.
- **FSM** (Morazán e Antunez, 2014) — **textual**: autômatos como código Racket, testáveis e versionáveis; muitos formalismos; execução **só em texto**, visualização como trabalho futuro.

Análise comparativa

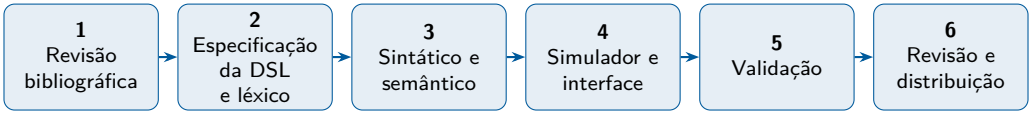
Trabalho	Descrição textual	Versiona-mento	Compilador/ diagnóstico	Simulação visual	Voltada ao ensino
JFLAP	Não	Não	Não	Sim	Sim
GAM	Não	Não	Não	Sim	Sim
Simulador UFSC	Não	Não	Não	Sim	Sim
AutomatoGraph	Não	Não	Não	Parcial	Sim
FSM	Sim (interna)	Sim	Via Racket	Não	Sim
Autosim	**Sim (externa)**	**Sim**	**Sim**	**Sim**	**Sim**

As gráficas **visualizam sem texto**; a textual **descreve sem visualizar**.
A Autosim une as duas coisas, com diagnóstico de compilador.

Roteiro

1. Introdução
2. Fundamentação Teórica
3. Trabalhos Relacionados
4. Metodologia de Construção da Solução
5. Ferramentas para Implementação
6. Projeto da DSL
7. Implementação
8. Resultados e Discussões
9. Conclusão

Pesquisa aplicada, exploratória: pesquisa bibliográfica + desenvolvimento e verificação funcional de um artefato (Gil, 2002; Wazlawick, 2014).



Organização

Dupla com divisão código/texto; reuniões semanais com o orientador; devolutivas incorporadas ao texto e à ferramenta (versões 1.0.1 a 1.0.4).

Critérios de validação

Testes por fase (todos aprovados) · autômatos de referência (veredito = definição formal) · erros induzidos (detectados e localizados).

Roteiro

1. Introdução
2. Fundamentação Teórica
3. Trabalhos Relacionados
4. Metodologia de Construção da Solução
5. Ferramentas para Implementação
6. Projeto da DSL
7. Implementação
8. Resultados e Discussões
9. Conclusão

Por que Rust — e as bibliotecas adotadas

Requisitos atendidos

- **Binário nativo único**: sem instalar JVM ou interpretador.
- **Enumerações com `match` exaustivo**: *tokens*, AST e erros tratados por completo, verificado pelo compilador.
- **Ecossistema** para compiladores com bons diagnósticos.
- **Interface multiplataforma**: Linux, Windows e macOS.

Comparada a Python, Java e C++ (Seção 5.4). Custo: curva de aprendizado.

Biblioteca	**Uso**
Logos 0.16	analisador léxico
Chumsky 0.13	analisador sintático
Ariadne 0.5	diagnósticos de erro
Iced 0.14	interface gráfica
GitHub Actions	testes + pacotes para 3 sistemas

Roteiro

1. Introdução
2. Fundamentação Teórica
3. Trabalhos Relacionados
4. Metodologia de Construção da Solução
5. Ferramentas para Implementação
6. Projeto da DSL
7. Implementação
8. Resultados e Discussões
9. Conclusão

A linguagem: o mesmo autômato, em texto

```
alfabeto { 'a', 'b' }

automato AFD exemplo_afd {
    estados { q0, q1, q2 }
    inicial q0
    finais { q2 }
    transicoes {
        q0 → q1 com 'a'    q0 → q0 com 'b'
        q1 → q1 com 'a'    q1 → q2 com 'b'
        q2 → q2 com 'a'    q2 → q2 com 'b'
    }
}

simular exemplo_afd com "ab"    // aceita
simular exemplo_afd com "bbb"   // rejeita
```

- DSL **externa**, declarativa, em **português**.
- Cada elemento da quíntupla tem uma declaração própria.
- Arquivo `.asl`: alfabeto comum, autômatos e comandos `simular`.
- Texto puro: **versionável** com Git.

Gramática e decisões de projeto

Trecho da gramática em EBNF (completa na Seção 6.2):

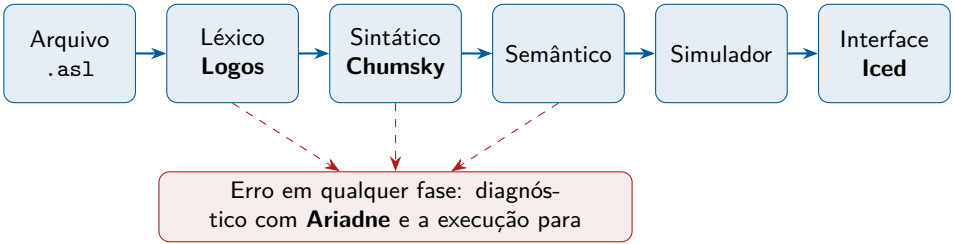
```
declaracao_automato ::= "automato" ("AFD" | "AFN") ident "{"
                        "estados" lista "inicial" ident "finais" lista
                        "transicoes" "{" transicao* "}" "}"
transicao           ::= ident "->" ident "com" (char_literal | "epsilon" | "eps")
comando_simulacao   ::= "simular" ident "com" cadeia_literal
```

- **Alfabeto único** por arquivo.
- Tipo **AFD/AFN explícito**: determinismo só é exigido do AFD.
- **Português** e **case-insensitive** (11 palavras reservadas).
- Transições ε com `epsilon` ou `eps`.

Roteiro

1. Introdução
2. Fundamentação Teórica
3. Trabalhos Relacionados
4. Metodologia de Construção da Solução
5. Ferramentas para Implementação
6. Projeto da DSL
7. Implementação
8. Resultados e Discussões
9. Conclusão

Pipeline de compilação

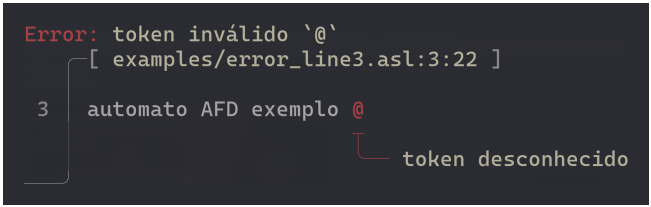


- **Logos** gera o analisador léxico, um AFD, a partir de anotações: um autômato lendo a linguagem de autômatos.
- **Chumsky** (*parser combinators*): a gramática é escrita compondo funções e produz a AST.

Análise semântica e diagnósticos

1. **Tabela de símbolos**: duplicatas; alfabeto vazio.
2. **Referências**: estados, símbolos e autômatos não declarados.
3. **Determinismo** (só AFD): sem ε e uma transição por (estado, símbolo).

Erros **acumulados**: todos aparecem em uma só execução.



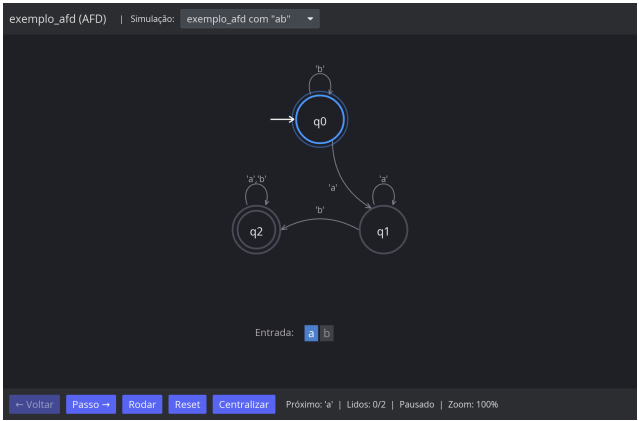
Diagnóstico real da Autosim: arquivo, **linha e coluna**, trecho realçado e descrição em português.

Simulador: um caminho, não uma nuvem de estados

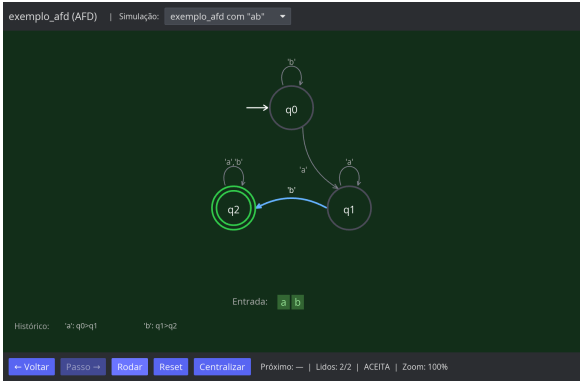
- No AFN, acender **todos os estados ativos** ao mesmo tempo dificultava acompanhar a execução.
- A Autosim exibe um **caminho único**:
  - cadeia aceita $\rightarrow$ um **caminho aceitador**;
  - cadeia rejeitada $\rightarrow$ o caminho que **mais se aproximou** da aceitação.
- Escolhido por **busca em largura** sobre (estado, símbolos consumidos): o veredito continua sendo o da definição formal.
- **Histórico completo**: avançar e voltar passo a passo.

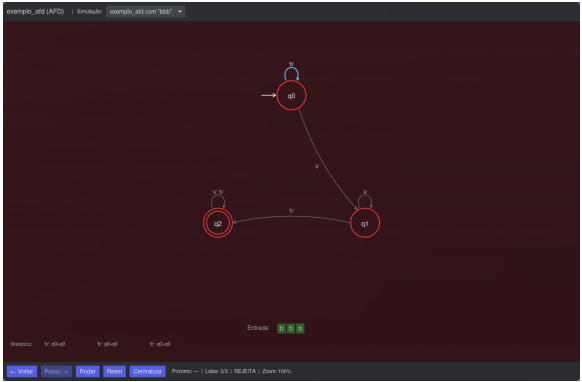
Interface gráfica (Iced)

- Arquitetura Elm (*Model-View-Update*).
- **Cabeçalho**: autômato e seletor de simulação.
- **Canvas**: diagrama, cadeia de entrada e histórico.
- **Controles**: Passo, Voltar, Rodar/Pausar (800 ms), Reset, Centralizar.
- *Layout* circular; raio cresce com o número de estados.
- Estado corrente em azul; no fim, verde ou vermelho.
- *Pan* e *zoom* (20%–400%); entrada e histórico ficam fixos.



Telas: aceitação e rejeição



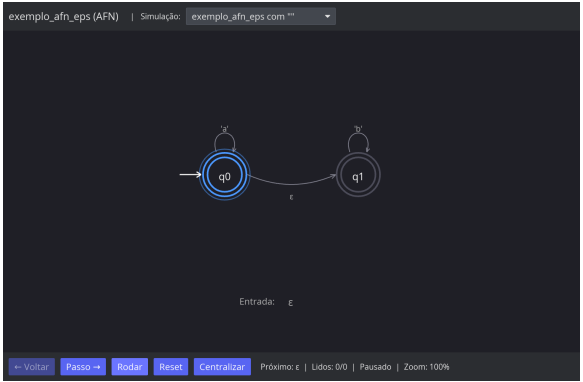


"ab": **ACEITA**, termina em q_2

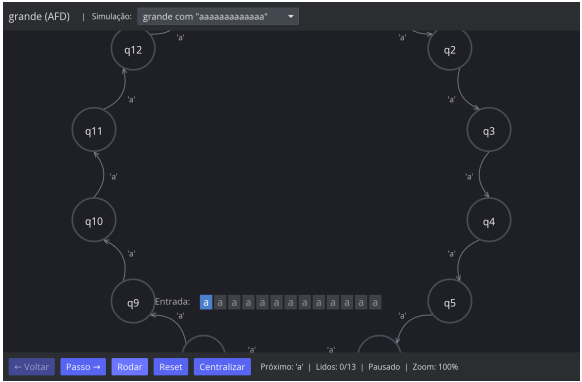
"bbb": **REJEITA**, termina em q_0

O mesmo AFD dos slides, que aceita as cadeias que contêm "ab".

Telas: AFN-ε e diagrama grande



AFN-ε para a^*b^*: a aresta ε liga q_0 a q_1



AFD de 14 estados: navegação por *pan* e *zoom*

Roteiro

1. Introdução
2. Fundamentação Teórica
3. Trabalhos Relacionados
4. Metodologia de Construção da Solução
5. Ferramentas para Implementação
6. Projeto da DSL
7. Implementação
8. Resultados e Discussões
9. Conclusão

Autosim em funcionamento

demonstração ao vivo

versão 1.0.4 · Linux

51 testes automatizados

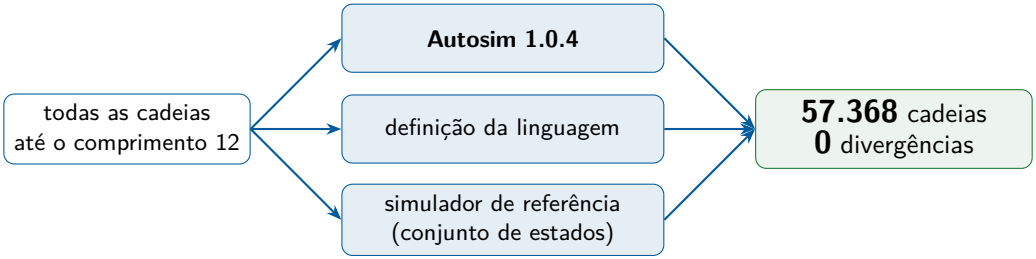
Analisador léxico	12
Analisador sintático	14
Analisador semântico	16
Simulador	9
Total	**51**

Todos aprovados; executados também antes de cada versão publicada.

6 arquivos com erros propositais

- Cada erro foi detectado **na fase correta**: léxica, sintática ou semântica.
- Toda mensagem traz **linha, coluna, trecho realçado** e descrição.
- Um arquivo com **5 erros semânticos** gerou as 5 mensagens em **uma única execução**.

Validação diferencial do simulador



- 8 autômatos clássicos (Hopcroft; Sipser): AFD, AFN e AFN-ε.
- Um AFN e o AFD obtido dele por subconjuntos aceitaram **as mesmas 4.092 cadeias**.
- Limite: exaustiva só até o comprimento testado; verifica o veredito, não o desenho do caminho.

Avaliação preliminar com estudantes

Percepção preliminar — fora do escopo da monografia

Levantamento feito após a entrega do texto, com o guia rápido da ferramenta. Não mede aprendizagem: indica como os estudantes perceberam a Autosim.

- **[PREENCHER]** participantes; período de aplicação.
- **[PREENCHER]** principais resultados (2 ou 3 perguntas).
- **[PREENCHER]** comentário que mais chamou atenção.

Discussão: os objetivos foram atendidos?

Objetivo	**Evidência**
1. DSL em EBNF	Todos os autômatos da validação escritos nela
2. Compilador	42 testes das 3 fases; 6 cenários de erro localizados
3. Simulador	9 testes + 57.368 simulações sem divergência
4. Interface	Demonstrada; **não** submetida a testes automatizados
5. Validação	Os 3 instrumentos aplicados; critérios atendidos

Limites

Casos de teste escritos pelos próprios autores · interface avaliada só pelo uso · a retroalimentação é **produzida corretamente**, mas o efeito sobre a **aprendizagem** não foi medido.

Roteiro

1. Introdução
2. Fundamentação Teórica
3. Trabalhos Relacionados
4. Metodologia de Construção da Solução
5. Ferramentas para Implementação
6. Projeto da DSL
7. Implementação
8. Resultados e Discussões
9. Conclusão

A pergunta de pesquisa foi respondida no **plano funcional**: a Autosim entrega retroalimentação imediata na construção e na execução de autômatos.

- **DSL** declarativa em português para AFDs e AFNs com ε, em EBNF.
- **Compilador pedagógico** com diagnósticos localizados.
- **Simulador** de caminho único, com histórico nos dois sentidos.
- **Interface gráfica** que desenha e anima o autômato a partir do texto.
- Validação: **51 testes**, **57.368 simulações** sem divergência e cenários de erro.
- Distribuída para Linux, Windows e macOS (versão 1.0.4), com manual de uso.

Limitações e trabalhos futuros

Limitações

- Alfabeto único por arquivo.
- Sem modo de linha de comando.
- Um arquivo por execução.
- Sem editor embutido com realce.
- Sem conversão AFN→AFD nem minimização.

Trabalhos futuros

- Editor com realce na interface.
- Conversões AFN→AFD, minimização e expressões regulares.
- Autômatos com pilha e **máquinas de Turing**.
- **Estudos controlados com estudantes**.

Obrigado!

Perguntas?

João Thomaz Vieira · Klaus Siegfried Beckmann

Orientador: Prof. Diógenes Cogo Furlan · UTP, 2026

Perguntas prováveis da banca

Proposta e contribuição

- **P1** Como sabem que a ferramenta ajuda o aluno?
- **P2** Qual é a contribuição original?
- **P3** O que a Autosim faz que o JFLAP não faz?
- **P4** Por que DSL externa, e não biblioteca como a FSM?

Linguagem e teoria

- **P5** A gramática é LL(1)?
- **P6** O AFD exige função de transição total?
- **P7** E se um AFD for declarado como AFN?
- **P8** E se a cadeia tiver símbolo fora do alfabeto?
- **P9** Por que um alfabeto único por arquivo?
- **P10** Por que a sintaxe em português?

Implementação

- **P11** Por que Rust, e não Python ou Java?
- **P12** Por que um caminho, e não o conjunto de estados?
- **P13** Funciona com autômatos grandes?

Validação, escopo e trabalho

- **P14** Como garantem que o simulador está correto?
- **P15** Por que não há conversão AFN→AFD?
- **P16** Como o estudante instala e usa?
- **P17** Como o trabalho foi dividido na dupla?

▸ Outros slides de apoio

P1. Como sabem que a ferramenta ajuda o aluno?

Resposta curta

Não afirmamos isso. A validação é **funcional**: mostra que a retroalimentação é **produzida corretamente**. O efeito sobre a aprendizagem é uma **hipótese**, a ser verificada em estudo com turmas.

- O recorte está declarado desde os objetivos (Cap. 1) e nas limitações (Seção 8.9).
- Um estudo controlado exigiria turmas comparáveis, pré e pós-teste e um semestre letivo: fora do prazo de um TCC.
- O formulário aplicado com estudantes é uma **percepção preliminar**, fora da monografia: indica aceitação, não aprendizagem.
- A motivação vem da literatura: falta de retroalimentação imediata (Morazán e Antunez, 2014).

◂ Perguntas

P2. Qual é a contribuição original?

Resposta curta

Compiladores e simuladores existem; a contribuição é o **acoplamento**: o estudante escreve texto, um compilador aponta os erros no vocabulário dos autômatos, e o diagrama é gerado e animado a partir desse texto.

- Entre as ferramentas analisadas, as gráficas não têm texto nem diagnóstico, e a textual (FSM) não visualiza.
- Wazlawick (2014): um novo artefato deve mostrar **em que difere** das soluções existentes (análise comparativa) e **verificar que funciona** (Cap. 8).
- Contribuições técnicas: a DSL em português com EBNF, o simulador de caminho único e a validação diferencial reprodutível.

◂ Perguntas

P3. O que a Autosim faz que o JFLAP não faz?

Resposta curta

A Autosim descreve o autômato em **texto**, versionável, e o valida com um **compilador** que localiza os erros. O JFLAP é muito mais abrangente; as duas são **complementares**.

Autosim

- Arquivo de texto: Git, *diff*, reaproveitamento.
- Erros com linha, coluna e descrição.
- Determinismo verificado quando o aluno declara AFD.
- Binário nativo, sem instalar Java.

JFLAP

- Toda a hierarquia de Chomsky (pilha, Turing, gramáticas).
- Conversão AFN→AFD, minimização, expressões regulares.
- Décadas de uso e maturidade.

◂ Perguntas

P4. Por que DSL externa, e não biblioteca (FSM)?

Resposta curta

Uma DSL externa tem **só a sintaxe do domínio**: o estudante não precisa aprender a linguagem hospedeira, e o compilador fala em estados, transições e alfabeto.

- Na FSM, o autômato é escrito na sintaxe de Racket; na Autosim, só com as construções do próprio domínio.
- A gramática restrita permite **verificações específicas**: estado não declarado, símbolo fora do alfabeto, determinismo do AFD.
- Custo: é preciso manter um compilador próprio, e a linguagem é menos expressiva (a FSM permite programar com os autômatos, compor, gerar testes aleatórios).

◂ Perguntas

P5. A gramática é LL(1)?

Resposta curta

A **linguagem** é LL(1). A EBNF, **como está escrita**, não é: na lista com vírgula final opcional, ao ver "**,**" é preciso olhar o próximo *token* (outro item ou "}") para decidir.

- Fatorando à esquerda, a decisão passa a usar um único *token*:

```
lista ::= "{" [ item resto ] "}"
resto ::= [ "," [ item resto ] ]
```

- O Chumsky não exige gramática LL(1): os combinadores tentam as alternativas em ordem.

◂ Perguntas

P6. O AFD da Autosim exige função de transição total?

Resposta curta

Não. Transições ausentes são permitidas: quando falta a transição, a simulação para e **rejeita**. Equivale a um **estado de rejeição implícito**, e a linguagem reconhecida não muda.

- O estado implícito é não final e tem laço para todos os símbolos; acrescentá-lo torna δ total sem alterar a linguagem.
- É a convenção usual nos diagramas dos livros, e poupa o aluno de desenhar o estado de "lixo".
- O que a Autosim exige do AFD é o que **contradiz** o determinismo: transição ε ou duas saídas para o mesmo (estado, símbolo).
- Coberto pelo teste "rejeição por falta de transição" (Seção 8.5).

◂ Perguntas

P7. E se um autômato determinístico for declarado como AFN? Seção 6.4.2

Resposta curta

É aceito: **todo AFD é um AFN** em que cada transição leva a um único estado. A classificação explícita serve para exigir determinismo quando o aluno **afirma** que o autômato é um AFD.

- Declarar AFD é uma afirmação que o compilador verifica: se houver ε ou duas saídas com o mesmo símbolo, o erro aponta as duas transições.
- Assim o erro conceitual "achei que era determinístico" vira um diagnóstico localizado.

◂ Perguntas

P8. E se a cadeia simulada tiver um símbolo fora do alfabeto?

Resposta curta

Hoje a cadeia do `simular` **não é conferida** contra o alfabeto: o símbolo não tem transição, e a cadeia é **rejeitada**, sem mensagem específica.

- O veredito está correto: uma cadeia com símbolo fora de Σ não pertence a $L(M) \subseteq \Sigma^*$.
- Mas o diagnóstico seria mais útil: a verificação natural é "símbolo 'z' fora do alfabeto" na análise semântica, como já ocorre nas **transições**.
- Reconhecemos como uma **melhoria** a fazer.

◂ Perguntas

P9. Por que um alfabeto único por arquivo?

Resposta curta

Para reforçar o conceito de Σ como conjunto declarado antes dos autômatos e ter **um único conjunto de referência** na verificação semântica.

- Todo símbolo usado em transição é conferido contra esse conjunto.
- É uma **limitação reconhecida**: autômatos com alfabetos distintos exigem arquivos separados.
- Trabalho futuro: alfabeto por autômato.

◂ Perguntas

Resposta curta

Para reduzir a barreira linguística e manter o foco nos conceitos: as palavras-chave são os próprios termos usados em aula (`estados`, `inicial`, `finais`, `transicoes`).

- As mensagens de erro também são em português e usam o vocabulário do domínio.
- Palavras-chave sem acento e sem diferenciar maiúsculas: `AFD`, `afd` e `Afd` são iguais.
- Contraponto: as linguagens de programação usam inglês; como a DSL é restrita ao ensino de autômatos, priorizamos a proximidade com a aula.

◂ Perguntas

P11. Por que Rust, e não Python ou Java?

Resposta curta

Por quatro requisitos: **binário nativo** sem ambiente de execução, **enumerações com `match` exaustivo** para AST e erros, **bibliotecas de diagnóstico** (Chumsky, Ariadne) e **interface multiplataforma** (Iced).

- Python e Java exigiriam interpretador ou JVM, ou empacotar o ambiente junto.
- C++ gera binário nativo, mas sem gerência de memória segura nem gerenciador de pacotes padrão.
- Custos aceitos: curva de aprendizado do *ownership*, compilação mais lenta e Iced ainda antes da versão 1.0.

◂ Perguntas

P12. Por que mostrar um caminho, e não o conjunto de estados?

Resposta curta

Com vários estados acesos ao mesmo tempo, a execução ficava **difícil de acompanhar**. Um caminho único é legível, e a escolha garante que ele seja **aceitador** sempre que a cadeia pertence à linguagem.

- Busca em largura sobre (estado, símbolos consumidos): vence quem consumiu mais e está mais perto de um estado final.
- O veredito é sempre o da definição formal: a busca explora todas as configurações.
- **Ciclos de** ε não travam: cada par é visitado uma vez.
- Detalhes: como o caminho é escolhido.

◂ Perguntas

P13. A ferramenta funciona com autômatos grandes?

Resposta curta

A **simulação**, sim: a busca visita no máximo $|Q| \cdot (n+1)$ pares, um custo polinomial. O limite está no **desenho**: o arranjo circular perde legibilidade com muitos estados.

- Na validação, 8.191 cadeias de um AFN foram simuladas em frações de segundo.
- O diagrama foi testado com um AFD de 14 estados: o raio cresce com o número de estados, e há *pan* e *zoom*.
- Para autômatos de dezenas de estados, um algoritmo de desenho de grafos reduziria cruzamentos de arestas: possível melhoria.

◂ Perguntas

P14. Como garantem que o simulador está correto?

Resposta curta

Por **validação diferencial**: 57.368 cadeias, todas as possíveis até o comprimento 12 em 8 autômatos clássicos, comparadas com a definição da linguagem e com um simulador de referência independente. **Zero divergências.**

- Além disso, 9 testes do simulador verificam a **forma** do caminho (contíguo, parte do inicial, usa transições existentes).
- O verificador foi testado: com uma definição alterada de propósito, as divergências apareceram na hora.
- Limite: exaustiva só até o comprimento testado; o código está em `validacao/`.

◂ Perguntas

P15. Por que não há conversão AFN→AFD?

Resposta curta

Por **escopo**: já na proposta do PG I, conversões e minimização ficaram de fora. A prioridade foi entregar o ciclo completo: texto, compilador com diagnósticos, simulação e visualização.

- As conversões estão nos **trabalhos futuros** (Cap. 9), junto com expressões regulares, autômatos com pilha e máquinas de Turing.
- A arquitetura facilita: a conversão operaria sobre a AST já validada, e o resultado poderia ser desenhado pela mesma interface.
- A equivalência AFN/AFD aparece na validação: um AFN e o AFD obtido por subconjuntos aceitaram as mesmas 4.092 cadeias.

◂ Perguntas

Resposta curta

Baixa o pacote pronto da versão 1.0.4 para **Linux, Windows ou macOS**, extrai e executa: **não precisa instalar o Rust**. Os Apêndices A a D são o manual.

- Pacotes gerados pelo GitHub Actions, que roda os testes antes de publicar.
- Uso: `autosim --path arquivo.asl` abre a interface com o diagrama e as simulações do arquivo.
- Quem tem Rust pode instalar com `cargo install --git` a partir do repositório.
- O manual cobre instalação, interface, escrita na DSL e interpretação dos diagnósticos.

◂ Perguntas

P17. Como o trabalho foi dividido na dupla?

Resposta curta

A **implementação** concentrou-se no Klaus, e a **monografia** foi dividida entre os dois. As decisões de projeto foram discutidas nas reuniões semanais com o orientador.

- Ferramenta e monografia em repositórios Git separados: cada um acompanhava as mudanças do outro, e o texto era conferido contra o código.
- Na revisão final, divergências entre texto e código foram corrigidas na ferramenta (versões 1.0.1 a 1.0.4).

◂ Perguntas

Outros

Fundamentação e ferramentas

- Hierarquia de Chomsky (2.1.2)
- Construção de subconjuntos (2.1.5)
- Diagnóstico de erros e DSL no ensino (2.2.8–2.2.9)
- *Ownership*, *borrowing* e Cargo (5.2–5.3)
- *Parser combinators*: nom, pest, chumsky (5.5)
- Versionamento e distribuição (5.7)

Projeto da DSL

- Palavras reservadas, ε e comentários (6.3, 6.5, 6.6)

Implementação

- Organização do código-fonte (7.1)
- Análise léxica e sintática (7.2–7.3)
- Como o caminho do AFN é escolhido (7.5.2)

Resultados e conclusão

- Testes por categoria (8.1–8.5)
- Simulações com autômatos clássicos (8.6)
- Dificuldades encontradas (Cap. 9)

▸ Perguntas prováveis da banca

Tipo	Linguagens	Regras	Modelo computacional
0	Recursivamente enumeráveis	$\alpha \to \beta$	Máquina de Turing
1	Sensíveis ao contexto	$\alpha A \beta \to \alpha \gamma \beta$	Autômato linearmente limitado
2	Livres de contexto	$A \to \gamma$	Autômato com pilha
3	**Regulares**	$A \to aB$ ou $A \to a$	**Autômato finito**

- A Autosim cobre o **tipo 3**: AFDs e AFNs reconhecem exatamente as linguagens regulares.
- Exemplo fora do alcance: $\{a^n b^n \mid n \geq 0\}$ é livre de contexto e exige pilha.
- Trabalhos futuros: subir a hierarquia com **autômatos com pilha** (tipo 2) e **máquinas de Turing** (tipo 0), com fita e cabeçote animados.

◂ Índice

Construção de subconjuntos (AFN → AFD)

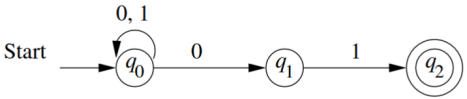
Cada estado do AFD é um **conjunto** de estados do AFN:

- $q_{0,D} = \{q_{0,N}\}$;
- $\delta_D(S, a) = \bigcup_{p \in S} \delta_N(p, a)$;
- S é final se $S \cap F_N \neq \varnothing$.

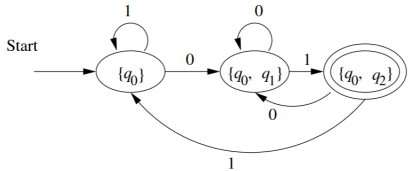
Só se constroem os subconjuntos **alcançáveis**: das $2^3 = 8$ possibilidades, o exemplo usa 3.

Pior caso: 2^n estados. AFD e AFN reconhecem a mesma classe de linguagens.

AFN: cadeias sobre $\{0, 1\}$ terminadas em 01



AFD equivalente



Fonte: Hopcroft, Motwani e Ullman (2006)

Índice

Diagnóstico de erros e DSL no ensino

Três critérios de um bom diagnóstico

1. **Localização precisa**: linha e coluna.
2. **Mensagem clara**: o que se esperava e o que foi encontrado, no vocabulário do domínio.
3. **Vários erros por execução**: sem repetir o ciclo a cada defeito.

Baseados nos objetivos de Aho *et al.* (2008).

DSL no ensino (Morazán e Antunez, 2014)

- O autômato vira um **objeto computacional**, escrito, testado e versionado como código.
- Aproxima linguagens formais da programação.
- Os próprios autores apontam como trabalho futuro da FSM **desenhar e animar** as máquinas escritas em código: é o que a Autosim faz.

◂ Índice

Memória sem coletor de lixo

- ***Ownership***: cada valor tem um único dono e é liberado quando ele sai de escopo.
- ***Borrowing***: várias referências `&T` **ou** uma `&mut T`, verificado na compilação.
- Na Autosim: as fases leem a AST por referência, sem cópias.

Tipos e ferramentas

- `enum` + `match` **exaustivo**: todo *token*, nó da AST e erro é tratado, ou o código não compila.
- **Cargo**: dependências (`Cargo.toml`, `crates.io`), compilação e testes (`cargo test`) em uma só ferramenta.

◂ Índice

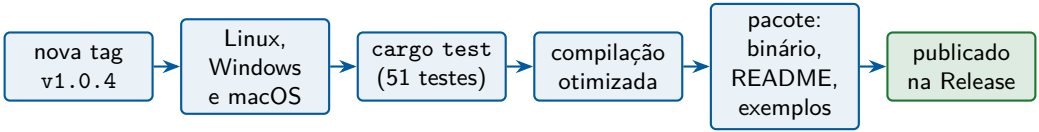
Combinadores: a gramática é escrita no próprio Rust, **compondo funções** pequenas, uma por construção da linguagem.

Critério	**nom**	**pest**	**chumsky**
Abordagem	Combinadores	Gramática PEG externa	Combinadores
Qualidade dos erros	Baixa	Média	**Alta**
Recuperação de erros	Manual	Limitada	**Nativa**
Localização	Manual	Automática	Automática
Maturidade	Alta	Alta	Média

Escolha: **chumsky**, pelo foco em diagnósticos (vários erros por execução, com localização) e por manter a gramática no código, sem geração externa.

◂ Índice

Versionamento e distribuição



- Código e monografia no **Git**, em repositórios no GitHub; o histórico serviu para reconstituir as etapas da Metodologia.
- **GitHub Actions**: os testes rodam antes da compilação, então um sistema só recebe pacote se todos os testes passarem nele.
- O estudante baixa o executável pronto, **sem instalar o Rust** (Apêndice A).
- Versões 1.0.1 a 1.0.4: correções vindas da revisão (alfabeto vazio, localização em *bytes*, uma mensagem de erro e um exemplo).

◂ Índice

Palavras reservadas, transições ε e comentários

Palavra	Função
`alfabeto`	declara o alfabeto
`automato`	declara um autômato
`AFD` / `AFN`	tipo do autômato
`estados`	conjunto de estados
`inicial`	estado inicial
`finais`	estados de aceitação
`transicoes`	bloco de transições
`simular`	comando de simulação
`com`	símbolo da transição
`epsilon` / `eps`	transição vazia

11 palavras, sem distinguir maiúsculas e minúsculas.

AFN-ε para a^*b^*:

```
alfabeto { 'a', 'b' }
// comentario de linha
automato AFN exemplo_afn_eps {
    estados { q0, q1 }
    inicial q0
    finais { q0, q1 }
    transicoes {
        q0 → q0 com 'a'
        q0 → q1 com eps
        q1 → q1 com 'b'
    }
}
```

Cada transição ε é um **passo próprio** da simulação, que muda o estado sem consumir símbolo. Comentários: só `//`, de linha.

◂ Índice

Organização do código-fonte

Caminho	Conteúdo
`src/main.rs`	lê e valida o arquivo, exibe os diagnósticos e abre a interface
`src/cli.rs`	argumentos de linha de comando
`src/lexer/`	*tokens* (`token.rs`) e erros léxicos
`src/parser/`	AST (`ast.rs`), gramática (`grammar.rs`) e erros sintáticos
`src/sema/`	tabela de símbolos (`table.rs`), verificações e erros semânticos
`src/runtime/`	escolha do caminho (`path.rs`) e execução passo a passo
`src/ui/`	diagrama, controles da simulação e navegação
`tests/`	um arquivo por categoria: `lexer`, `parser`, `sema`, `runtime`
`examples/`	programas na DSL, válidos e com erros propositais
`.github/workflows/`	geração dos pacotes de distribuição

Cada fase do *pipeline* ocupa um módulo próprio, com seus testes.

◂ Índice

Análise léxica e sintática

Léxico (Logos)

- `#[derive(Logos)]` gera um **AFD** a partir das anotações do `enum Token`.
- Categorias: palavras reservadas (`ignore(case)`), símbolos (`{ } , ->`), literais (`'a'`, `"ab"`) e identificadores.
- Espaços e comentários são descartados.

Sintático (Chumsky)

- AST: `Program`, `Automaton`, `Transition`, `Simulation`.
- Mensagens no formato *esperava X, encontrou Y*: `esperava 'inicial', encontrou 'q0'`.

Localização (*spans*)

- Léxico: intervalo de ***bytes*** de cada *token*.
- AST: intervalo de **índices de *tokens***.
- Diagnóstico: volta a *bytes*; o Ariadne calcula linha e coluna.
- *Bytes*, e não caracteres: letras acentuadas ocupam mais de um *byte* em UTF-8.

```
Error: token inválido `@`
    [ examples/error_line3.asl:3:22 ]

 3  automato AFD exemplo @
                         ┬
                         ╰── token desconhecido
```

◂ Índice

Como o caminho do AFN é escolhido

1. **Busca em largura** sobre os pares (estado, símbolos consumidos), seguindo transições ε (não avançam) e com o próximo símbolo (avançam uma posição). Cada par é visitado uma vez: ciclos de ε não travam a busca.
2. Entre os pares alcançados, elege-se o melhor por:
   1. **maior** número de símbolos consumidos;
   2. **menor** distância até um estado final.
3. A distância vem de uma **segunda busca em largura**, no grafo invertido, a partir de todos os estados finais (calculada uma vez, antes).
4. O caminho é reconstruído pelos apontadores da busca e é o **mais curto** até aquele par.

Por que funciona

Se a cadeia pertence à linguagem, há um par que consumiu tudo e está em um estado final (distância 0): ele vence, e o caminho exibido é **aceitador**. Se não pertence, vence o caminho que chega mais longe e termina mais perto de um final. O veredito é sempre o da definição formal.

◂ Índice

Testes por categoria

Categoria	Qtd.	Principais verificações
Análise léxica	12	palavras reservadas, *case-insensitive*, literais, epsilon, comentários, erros
Análise sintática	14	*parsing* completo, AFN com epsilon, múltiplos autômatos, erros
Análise semântica	16	AFD e AFN válidos, alfabeto vazio, duplicatas, referências inválidas, determinismo
Simulação	9	caminho aceitador, epsilon, cadeia vazia, rejeição por falta de transição, navegação
Total	**51**	todos aprovados, com `cargo test`

◂ Índice

Simulações com autômatos clássicos

Autômato	Cadeia	Esperado	Obtido
AFD: contém 01	1101	Aceita	Aceita
AFD: contém 01	111	Rejeita	Rejeita
AFD: contém 01	101	Aceita	Aceita
AFD: contém 01	000	Rejeita	Rejeita
AFN: termina em 01	001	Aceita	Aceita
AFN: termina em 01	010	Rejeita	Rejeita
AFN-ε: a^*b^*	aaa	Aceita	Aceita
AFN-ε: a^*b^*	aabb	Aceita	Aceita
AFN-ε: a^*b^*	ba	Rejeita	Rejeita
AFN-ε: a^*b^*	ε	Aceita	Aceita

- Autômatos de Hopcroft e Sipser.
- Todos os vereditos conferem.
- Limite: poucas cadeias, escolhidas à mão. Por isso veio a **validação diferencial**: 57.368 cadeias, 0 divergências (Seção 8.7).

◂ Índice

Dificuldades encontradas

1. **AFN com vários estados acesos**: a primeira versão propagava o conjunto de estados e a execução ficava difícil de seguir. → Escolha prévia de um **caminho único** (Seção 7.5.2).
2. **Autômatos grandes**: estados se sobrepunham ou saíam da janela. → Raio do arranjo cresce com o número de estados, e ***pan* e *zoom***.
3. **Localização dos erros**: a AST guarda índices de *tokens*, e acentos em comentários deslocavam a coluna (*bytes* lidos como caracteres). → Conversão de volta ao texto e ***bytes* em toda a cadeia** de diagnóstico.
4. **Coerência entre texto, código e teoria**: a gramática e um teste aceitavam `alfabeto { }`, mas a definição exige alfabeto não vazio. → **Verificação semântica** específica.

◂ Índice